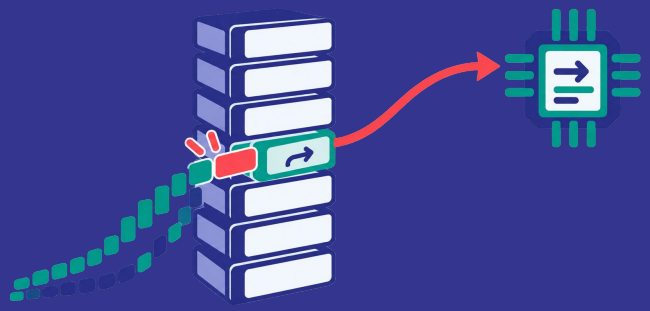
بسم الله الرحمن الرحیم

Data and Network Security

SESSION 04

Memory, the Call Stack, and Stack Smashing



Solmaz Salimi
Sharif University of Technology

s.salimi@sharif.edu

Last lecture and today’s lecture

Session 3	Access control models and end-to-end enforcement
Part 1	Memory as bytes and process regions
Part 2	Calls, registers, and stack frames
Part 3	Stack smashing and code injection
Part 4	Observe and reason with GDB
Checkpoint	Compute an offset, then verify assumptions








Running example: a vulnerable greeting service

```c
void greet(const char *name) {
    char buf[16];
    strcpy(buf, name);
    printf("Hello %s\n", buf);
}
```

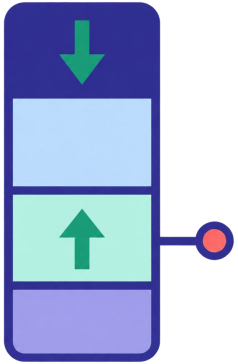
UNTRUSTED INPUT

What can the input change besides the string?

The answer depends on the concrete stack frame, not only on the C source.

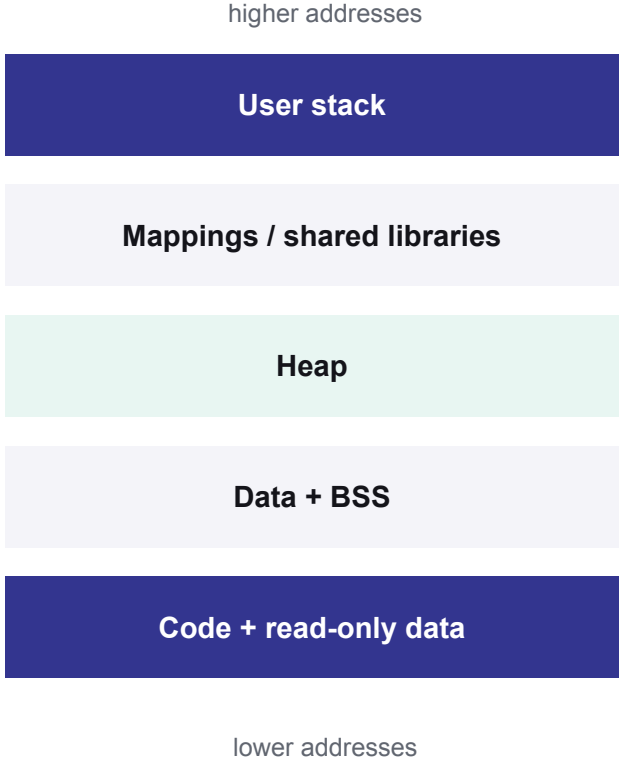
- Input length is attacker-controlled
- Destination capacity never reaches strcpy
- Adjacent memory may hold control data

Part 1: Memory as bytes and regions



A process address space has regions with different roles

- Code contains instructions; data/BSS holds globals
- malloc allocates from the heap
- Function-local state is commonly placed on the stack
- Mappings hold shared libraries and mapped files



Conceptual map: exact addresses and ordering vary; ASLR may randomize placement.

Bytes, words, and little-endian representation

1 byte = 8 bits · x86-64 pointer = 8 bytes

```
Address value: 0x00000000401234
```



increasing memory addresses →

Little-endian stores the least-significant byte at the lowest address.

Pointers and C strings carry no automatic bounds

POINTER

Stores an address.
It does not encode the size of the object at that address.

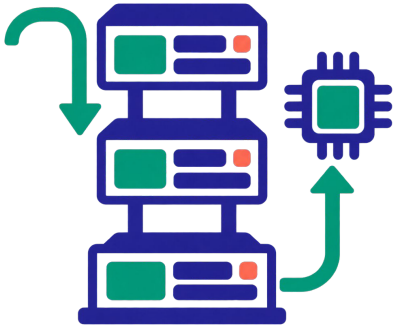
C STRING

A byte sequence ending at the first NUL byte (\0).
The terminator is data, not metadata.

strcpy(dst, src) copies until it reaches NUL in src. It receives no destination capacity.

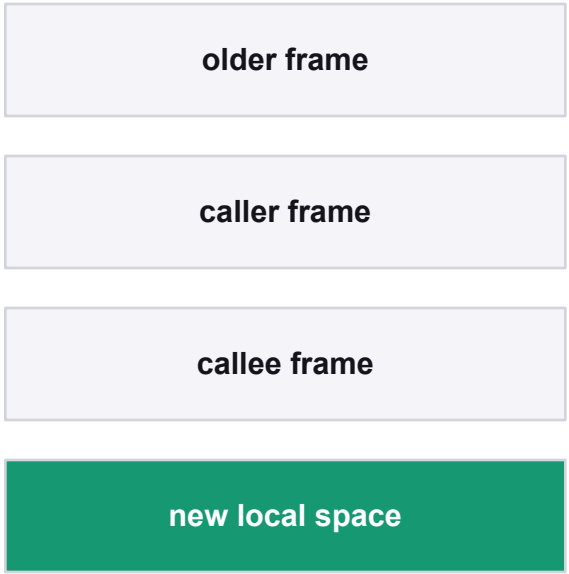
Safe use requires a checked length and a destination whose capacity is known at the copy site.

Part 2: Calls and stack frames



The stack is LIFO and typically grows downward

- push: lower RSP, then store a value
- pop: load a value, then raise RSP
- each call allocates a logical frame
- return removes the callee frame



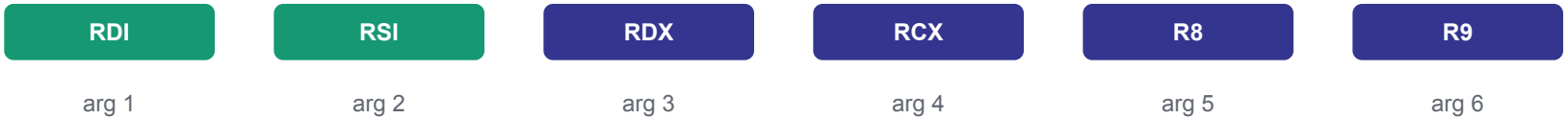
Important distinction: stack allocation moves downward; copying bytes into an array still advances toward higher addresses.

Four registers explain most of the call-stack story

RIP	**address of the next instruction**
RSP	current top of the stack
RBP	optional stable reference for one frame
RAX	integer / pointer return value

Modern compilers may omit RBP as a frame pointer; RSP and saved return addresses still matter.

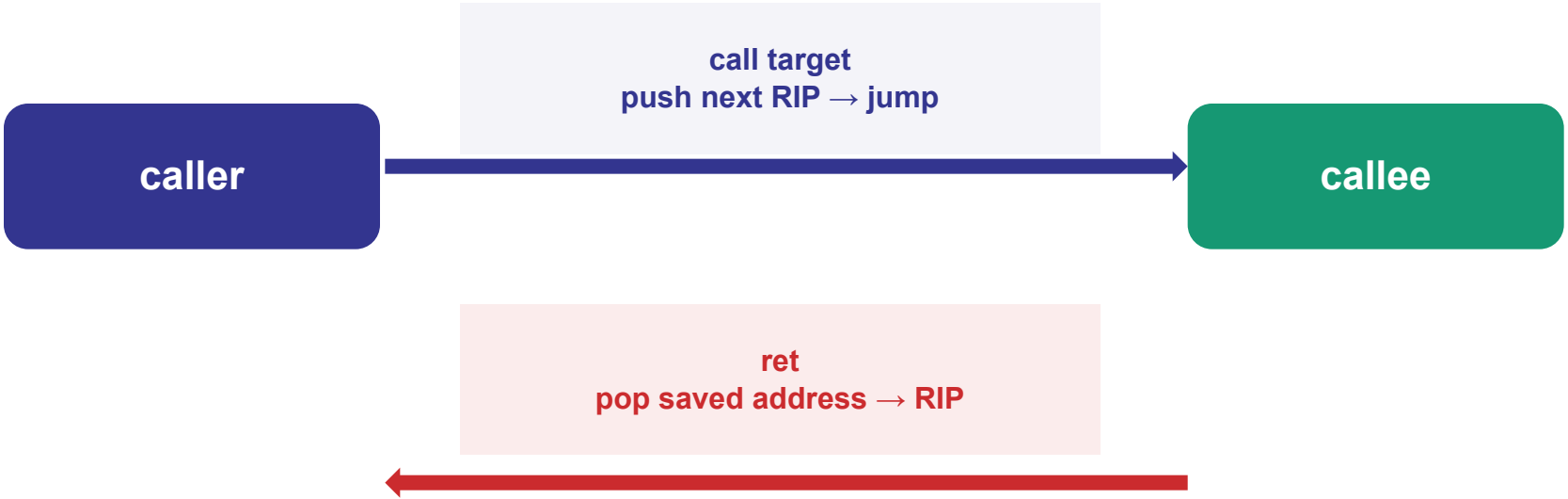
System V x86-64 passes the first arguments in registers



Additional arguments spill to the stack.

Caller	places arguments and preserves caller-saved registers when needed
Callee	preserves callee-saved registers it uses and restores the stack
Return	RAX carries a typical integer or pointer result

call saves a return address; ret trusts it

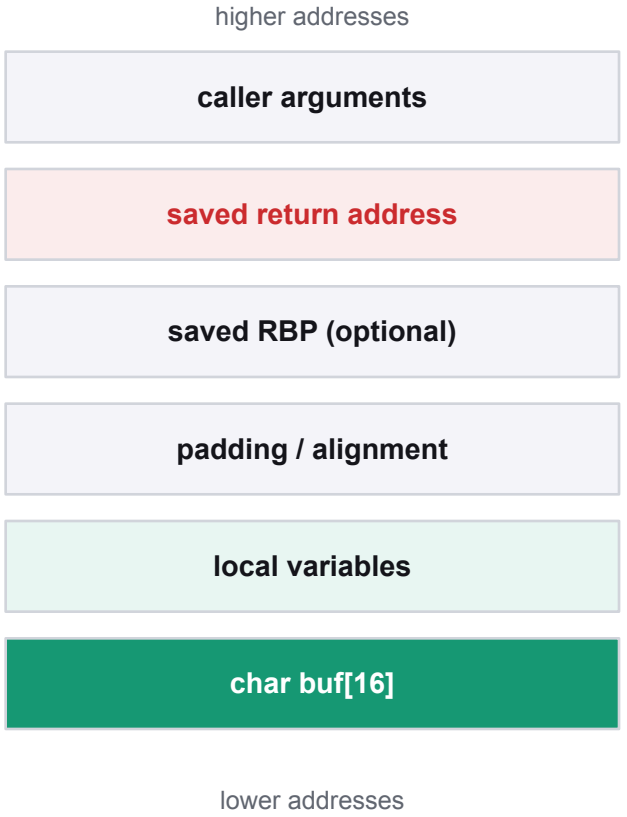


Security consequence: if memory corruption changes the saved return address, ret transfers control to the attacker-chosen value.

A stack frame mixes local data with control data

- Locals and return state share one writable region
- Padding changes offsets
- The compiler may reorder locals or omit RBP
- Disassembly; not source intuition; defines the frame

Exact layout is an implementation fact.



Worked trace: main → greet → main

1 Before call

RSP points into main
RIP executes the caller

RSP ↓

2 Inside greet

call saved return address
locals reserve stack space

RSP ↓↓↓

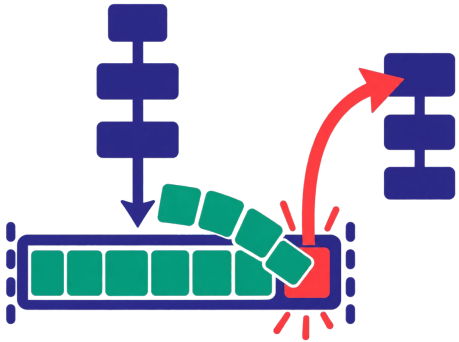
3 On return

frame is released
ret restores saved RIP

RSP ↑

Invariant: the return address must remain exactly the value written by call.

Part 3: Stack smashing



The bug is an unbounded write into a fixed-size object



```
void greet(const char *name) {
    char buf[16];
    strcpy(buf, name);
}
```

name — attacker chooses length

strcpy — copies until NUL

buf[16] — capacity is not checked

Root cause: the copy operation cannot enforce a bound it was never given.

gets is even worse: its interface has no buffer-length argument at all.

Stack growth and copy direction are different



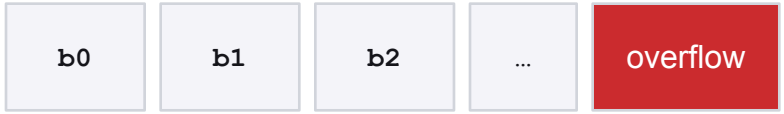
FRAME ALLOCATION

new frame

older frame

lower addresses ↓

BYTE COPY



increasing addresses →

The stack grows down only when space is allocated. An array write still advances through successive addresses.

Corrupting the saved return address changes the next instruction

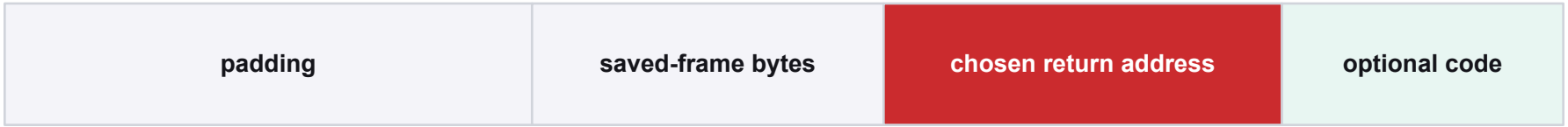


BEFORE	AFTER OVERFLOW
buf[16]	41 41 41 …
padding / saved RBP	41 41 41 …
saved return address	41 41 41 41 41 41 41 41

First result: RIP becomes 0x4141414141414141 → invalid control target → crash.

A classic payload has structure

```
payload = padding || saved-frame bytes || target address || optional machine code
```



The offset is measured from the compiled frame. It is not guaranteed by the C source.

Address bytes must match little-endian memory order

Chosen target: 0x00000000401234

34	12	40	00	00	00	00	00

bytes written in memory order

String-based inputs stop at NUL. The zero bytes in this address may terminate the copy before the full pointer is written.

Exploitability depends on the input channel, not only on the vulnerable instruction.

Historical code injection used shellcode and a NOP sled

- Injected instructions must be position-independent
- Forbidden bytes cannot appear in the input
- The return address can land anywhere in the NOP region



Historical model: modern systems add non-executable memory, randomization, canaries, and other defenses; covered in next sessions.

A crash is not automatically an exploit

✓	**Reach**	Does the write reach security-relevant data?
✓	**Control**	Can the attacker choose the overwritten bytes?
✓	**Target**	Is a useful destination known or recoverable?
✓	**Transport**	Do NULs, newlines, or encoding alter the input?
✓	**Mitigations**	Can the path survive defenses and runtime checks?

Exploitability is an end-to-end property of the bug, compiled layout, input path, and runtime environment.

Part 4: Observe and reason



GDB evidence

```
disassemble greet
info registers rsp rbp rip
x/24gx $rsp
bt
```

Disassembly	Find local allocation and call/ret instructions
Registers	Record concrete stack and instruction addresses
Memory	Locate the buffer and saved return address
Backtrace	Detect frame corruption after the crash

Checkpoint: compute, then verify

Simplified frame

```
16-byte buffer
8-byte saved RBP
8-byte saved return address
```

1 Nominal offset to the first return-address byte?

2 Little-endian bytes for 0x00000000401234?

3 Why must assembly confirm both answers?

Expected reasoning: 24 bytes nominally; 34 12 40 00 00 00 00 00; padding, optimization, and omitted frame pointers can change the real layout.

References

BERKELEY	CS 161 textbook, Ch. 2: x86 Assembly and Call Stack; Ch. 3.1–3.2: Buffer Overflow and Stack Smashing.
STANFORD	CS 155 Lecture 2: Control Hijacking Attacks; Project 1: Control Hijacking.
CLASSIC	Aleph One, “Smashing the Stack for Fun and Profit,” Phrack 49, 1996.
PRIOR TERM	Control-hijacking lecture and supplements

Next: advanced memory vulnerabilities

Session 5 moves beyond the classic stack overflow.

Integer conversion	**Signedness and arithmetic wraparound**
Format strings	**Reads and writes driven by format directives**
Heap corruption	**Metadata, objects, and indirect control targets**
Use-after-free	**Stale pointers after object lifetime ends**

Today: how control data sits next to bytes. Next: how other memory errors reach the same security boundary.